

UD 4 - RELACIONES ENTRE CLASES



UD 4 - RELACIONES ENTRE CLASES

Índice

1. Siempre en mente: el cambio como constante
2. Composición
3. Herencia
4. Polimorfismo
5. Interfaces

ENRIQUE ROYO SÁNCHEZ

UD 4 - RELACIONES ENTRE CLASES

1 - Siempre en mente: el cambio como constante

1 – Siempre en mente: el cambio como constante

- Recordemos que en el mundo del desarrollo de software solo existe una **CONSTANTE**:

“TODO CAMBIA”

- Como desarrolladores de software tenemos que MENTALIZARNOS DE QUE EL CAMBIO ES **INEVITABLE**. Así que lo mejor es **PREPARARSE PARA EL CAMBIO**. Esto implica tomar ciertas medidas en nuestro código que nos permitan “gestionar correctamente” el cambio.
- Establecer una arquitectura por capas y **repartir responsabilidades entre distintas clases para que colaboren entre sí** nos permitirá identificar más rápidamente qué pieza de código debe modificarse y qué cambios son necesarios.

¿Cómo se hace eso de que las clases colaboren entre sí?

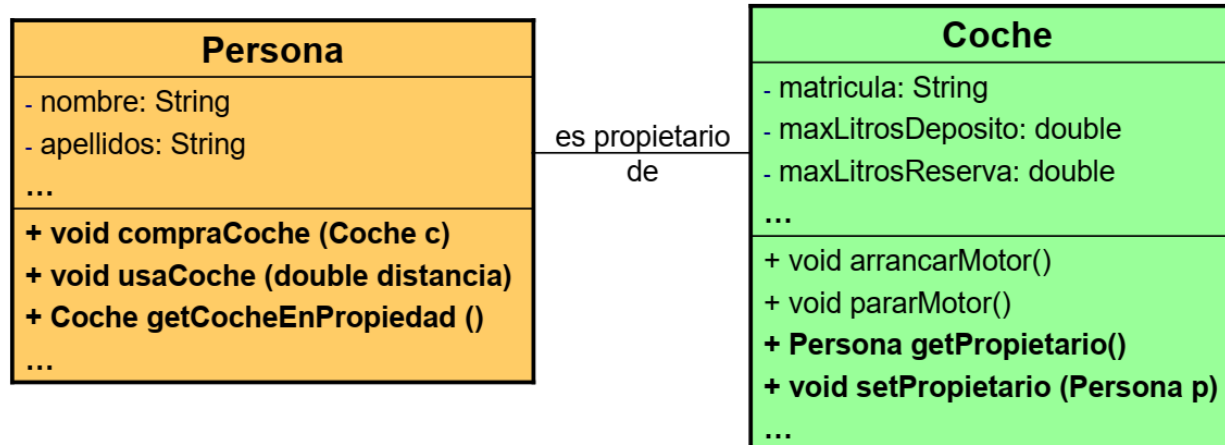


UD 4 - RELACIONES ENTRE CLASES

2 - Composición

2 – Composición

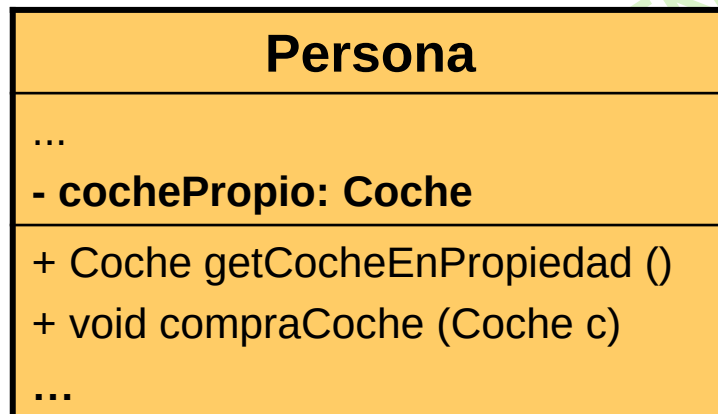
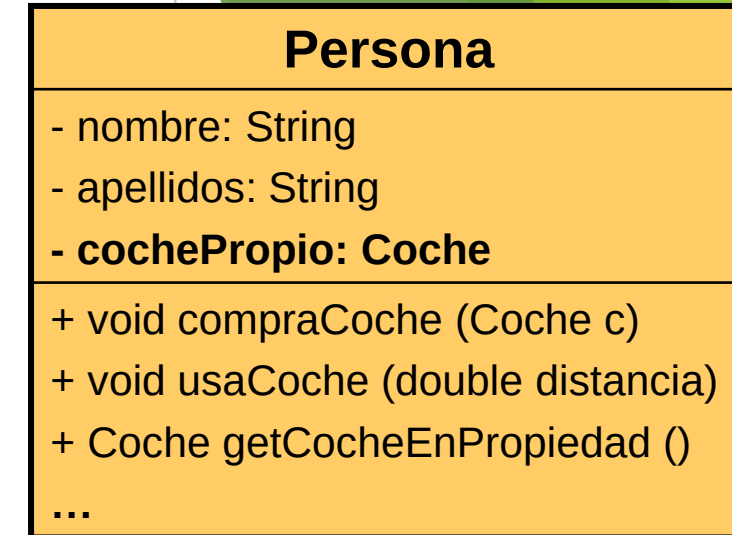
- El primer tipo de relación que vamos a estudiar es la composición, que es el más sencillo y el más utilizado.
- En este caso, se dice que las clases presentan una relación de tipo “está compuesto por”, “contiene”, “es propietario de”...
- Ejemplo:



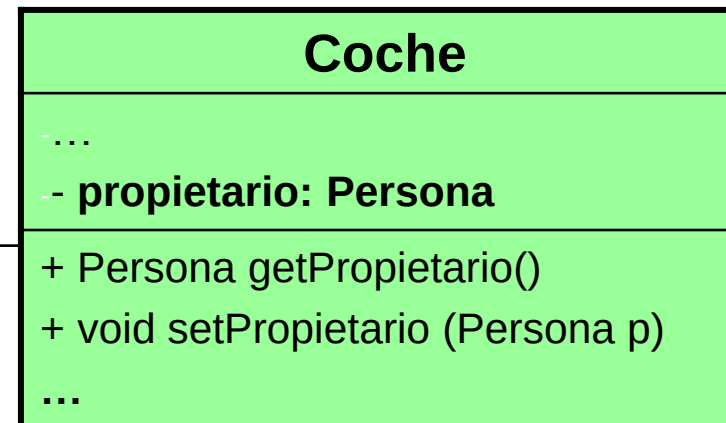
UD 4 - RELACIONES ENTRE CLASES

2 - Composición

- ▶ En estos casos, la forma de llevar esto al código consiste en que el objeto “poseedor o contenedor” guarde una referencia al “objeto poseído o contenido”.
- ▶ Con este modelo podríamos “**navegar**” desde un objeto Persona hasta su objeto coche y manipular dicho objeto, **pero no a la inversa**. Es decir, si yo tengo un objeto Coche, no podría conocer a su dueño... Sólo con añadir una referencia en la clase Coche a la Persona se arregla el problema:



es propietario de



Realizar el ejercicio
1 del boletín

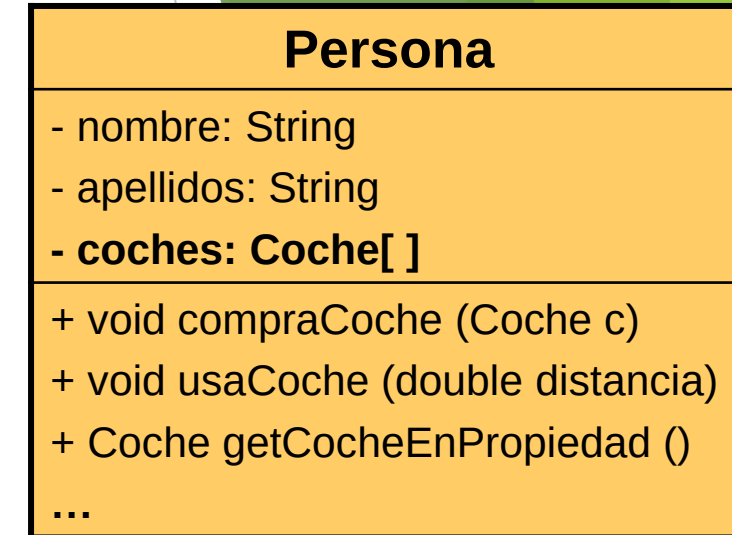
UD 4 - RELACIONES ENTRE CLASES

2 - Composición

- ▶ También nos podría interesar que una persona tenga varios coches y sería tan sencillo como añadir como propiedad una **colección de referencias a objetos coche** (con un array, por ejemplo).
- ▶ La composición es una técnica que se utiliza habitualmente. Además, está fuertemente relacionada con cómo se almacenan los datos de un objeto en una base de datos relacional.
- ▶ Ejemplo de tablas de base de datos que representan a dos clases relacionadas mediante composición.

Tabla persona		
dni	nombre	apellidos
27485658E	Juan	López Haro
87485159R	Ana	Smith
...	...	...

Tabla coche			
matricula	marca	modelo	dni_propietario
2342-RTF	Volvo	S30	27485658E
1264-RFZ	Seat	Ibiza	87485159R
...	...	...	



Realizar los
ejercicios 2 al 5 del
boletín

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

3 – Herencia

- ▶ Heredar algo, en el sentido general de la palabra, consiste en RECIBIR algo de alguien que, normalmente, tiene una relación de parentesco con el heredero.
- ▶ De algún modo los elementos heredados se REUTILIZAN. Así podemos hablar de la herencia de bienes y capitales o también de la herencia genética.
- ▶ En los lenguajes orientados a objetos existe el concepto de **herencia entre clases**, que es una potente herramienta de reutilización código.
- ▶ La reutilización de código nos permite desarrollar nuestras aplicaciones más rápidamente y mejorar su mantenimiento.

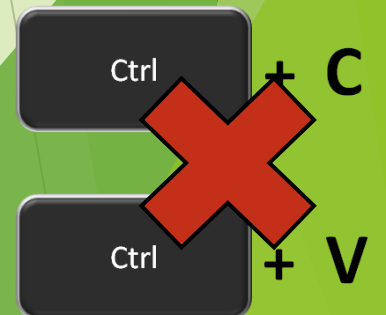


reduce
Reuse
RECYCLE

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- ▶ Imagina que tenemos una clase **Persona** que hemos ido perfeccionando con el tiempo y que es capaz de guardar mucha información (nombre, apellidos, DNI, pasaporte, nacionalidad, teléfonos, dirección postal, dirección mail, fecha nacimiento, nivel de estudios...)
- ▶ Ahora nos interesa hacer dos clases más: **Empleado** y **Cliente**.
 - ▶ Un Empleado “es una” Persona pero que además tiene una fecha de contratación y un salario.
 - ▶ Un Cliente “es una” Persona que además tiene una fecha de alta en el sistema y un histórico de compras.
- ▶ ¿Cómo podríamos reutilizar la clase Persona en las nuevas clases?
 - ▶ **Opción cutre – Copiar/Pegar:** provocaría una duplicidad del código de modo que si, en un futuro, hay que cambiar algo de la Persona tendría que acordarme de cambiarlo también en el Empleado y el Cliente... **MALA PRÁCTICA, es fácil cometer errores... Poco resistente al CAMBIO...**

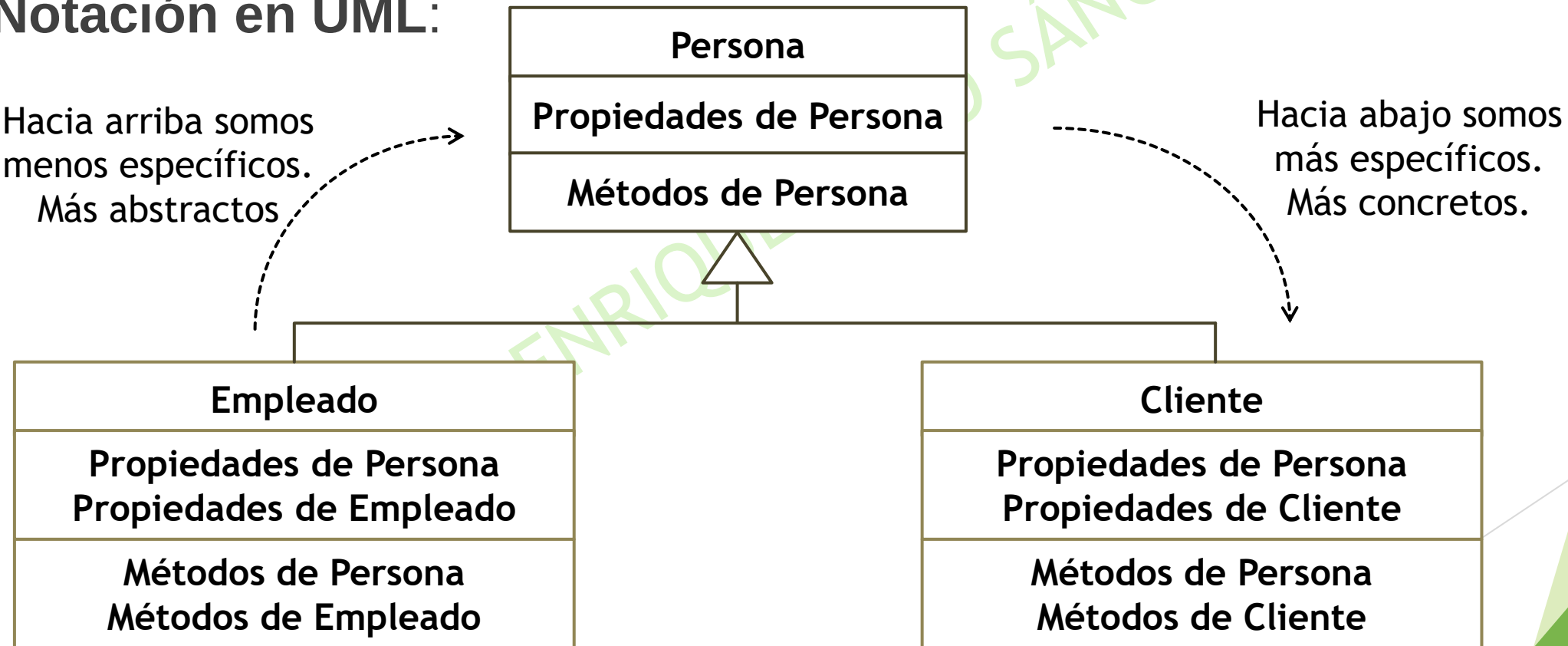


UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- **Opción ganadora – la HERENCIA:** la idea consiste en hacer que las clases Empleado y Cliente *hereden* de la clase Persona, de esta forma los cambios que hagamos en esta clase se propagan automáticamente a las clases herederas.

- **Notación en UML:**



La herencia es uno de los "conceptos estrella" del paradigma orientado a objetos

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- La sintaxis de la herencia en Java es muy sencilla:

```
public class Empleado extends Persona {  
    private double salario;  
    private LocalDate fechaContratacion;  
    // Getters y setters  
}
```

- De este modo tan sencillo conseguimos que la clase Empleado reciba toda la funcionalidad de la clase Persona.
- Decimos entonces que la clase Empleado “**extiende**” a la clase Persona o bien que “hereda” de esta. También se habla de “Superclase” y “subclases” o de clase base y clases derivadas o de clases madre/padre y clases “hijas”...

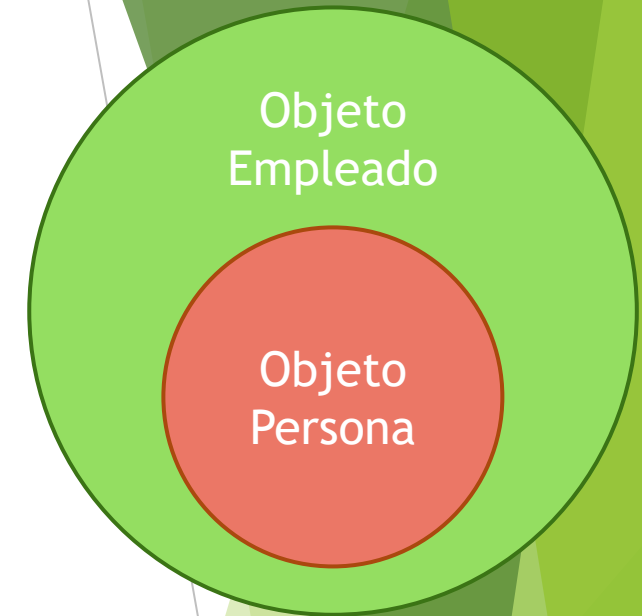
ENRIQUE ROYO SÁNCHEZ



UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- ▶ Cuando creamos un objeto de la clase Empleado tenemos que imaginar que éste tiene **un objeto Persona “encerrado” dentro**.
- ▶ El objeto Empleado podrá acceder o no a las propiedades y métodos de la clase Persona dependiendo de los **modificadores de acceso** que se hayan usado. Entonces:
 - ▶ Los **miembros public** de Persona serán accesibles por Empleado y por cualquier otro objeto de nuestra aplicación.
 - ▶ Los **miembros private** de Persona no serán accesibles ni por Empleado ni por cualquier otro objeto de la aplicación.
 - ▶ Los **miembros “friendly”** de Persona serán accesibles por las clases que compartan paquete con Persona, sin importar si hay herencia o no.
 - ▶ Los **miembros protected** de Persona serán accesibles por Empleado y también por aquellas clases que compartan paquete con Persona.



Cuando nos referimos a las propiedades y métodos de una clase indistintamente, hablamos de los **MIEMBROS DE LA CLASE**

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

► Tabla resumen:

Un miembro en la superclase definido como:	¿Es accesible por una subclase?	¿Es accesible por otra clase en el mismo paquete?	¿Es accesible por otra clase fuera del paquete?
private	NO	NO	NO
“friendly”	Sólo si ambas están en el mismo paquete	SI	NO
protected	SI	SI	NO
public	SI	SI	SI

REGLA GENERAL:

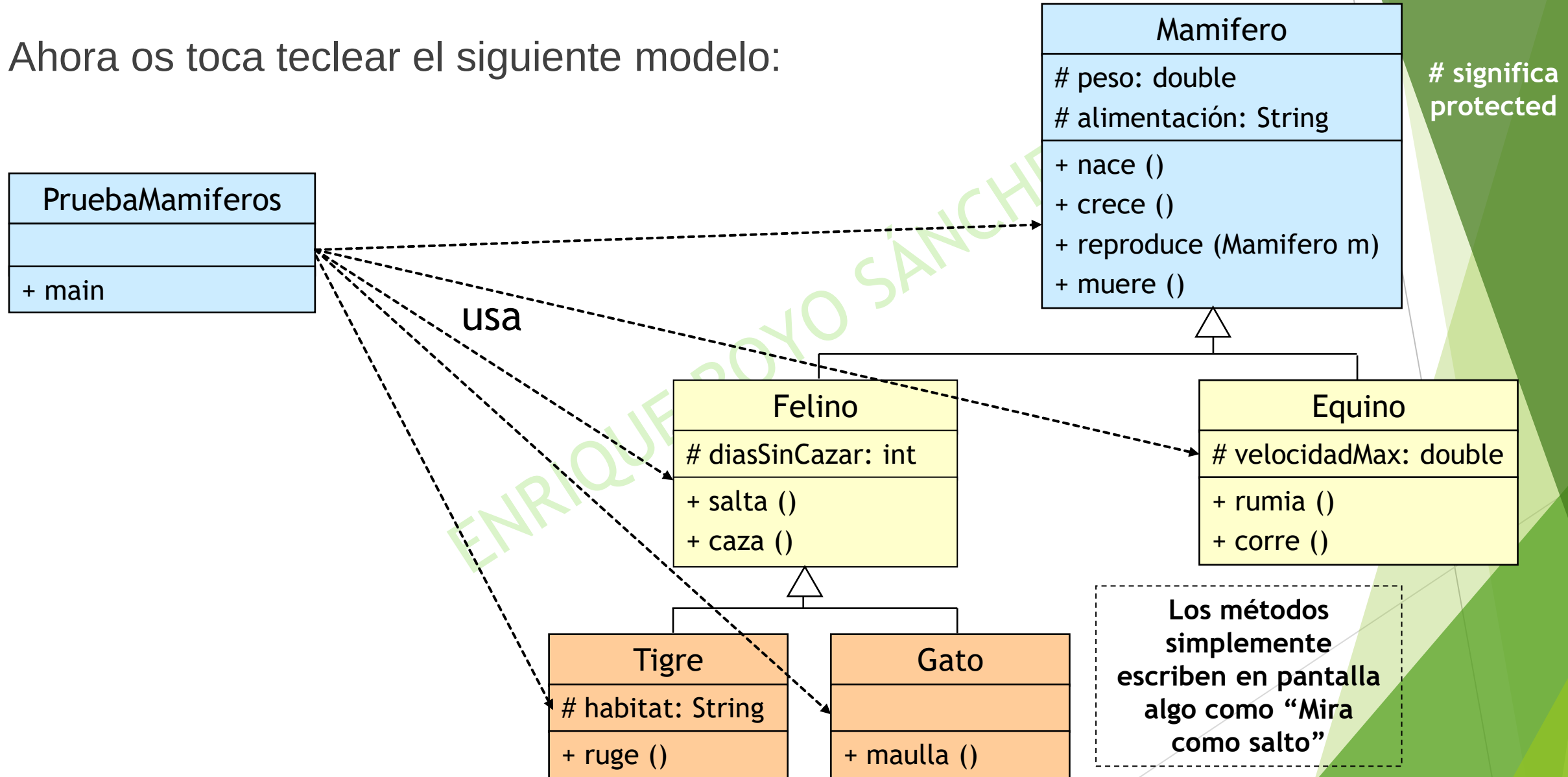
Las propiedades siempre deberían ser privadas porque representan el estado interno del objeto. El uso de cualquier otro modificador debería estar justificado.

Estudiamos el siguiente proyecto del repositorio:
U4.P1.Herencia

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- Ahora os toca teclear el siguiente modelo:



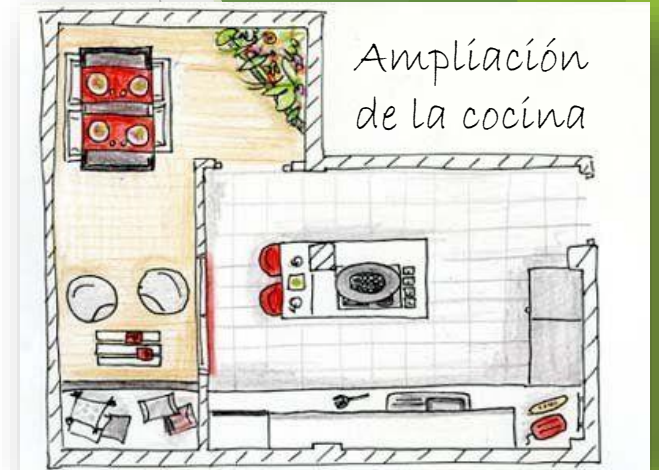
UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

► ¿Cuándo nos viene bien usar la herencia?

Siempre que haya una **relación tipo “es un/a”** entre dos clases y, además, nos encontremos en al menos una de estas situaciones:

- Nos interesa que la subclase hija **amplíe la funcionalidad** de la superclase, añadiendo nuevos miembros a los ya existentes. Los ejemplos anteriores reflejan esta situación.
- Nos interesa que la subclase **modifique el comportamiento de ciertos métodos** de la superclase “reescribiendo el código”. Decimos entonces que la subclase **sobrescribe** (**override** en inglés: anular, invalidar, desestimar) uno o varios métodos de la superclase. Veamos un ejemplo de esto:



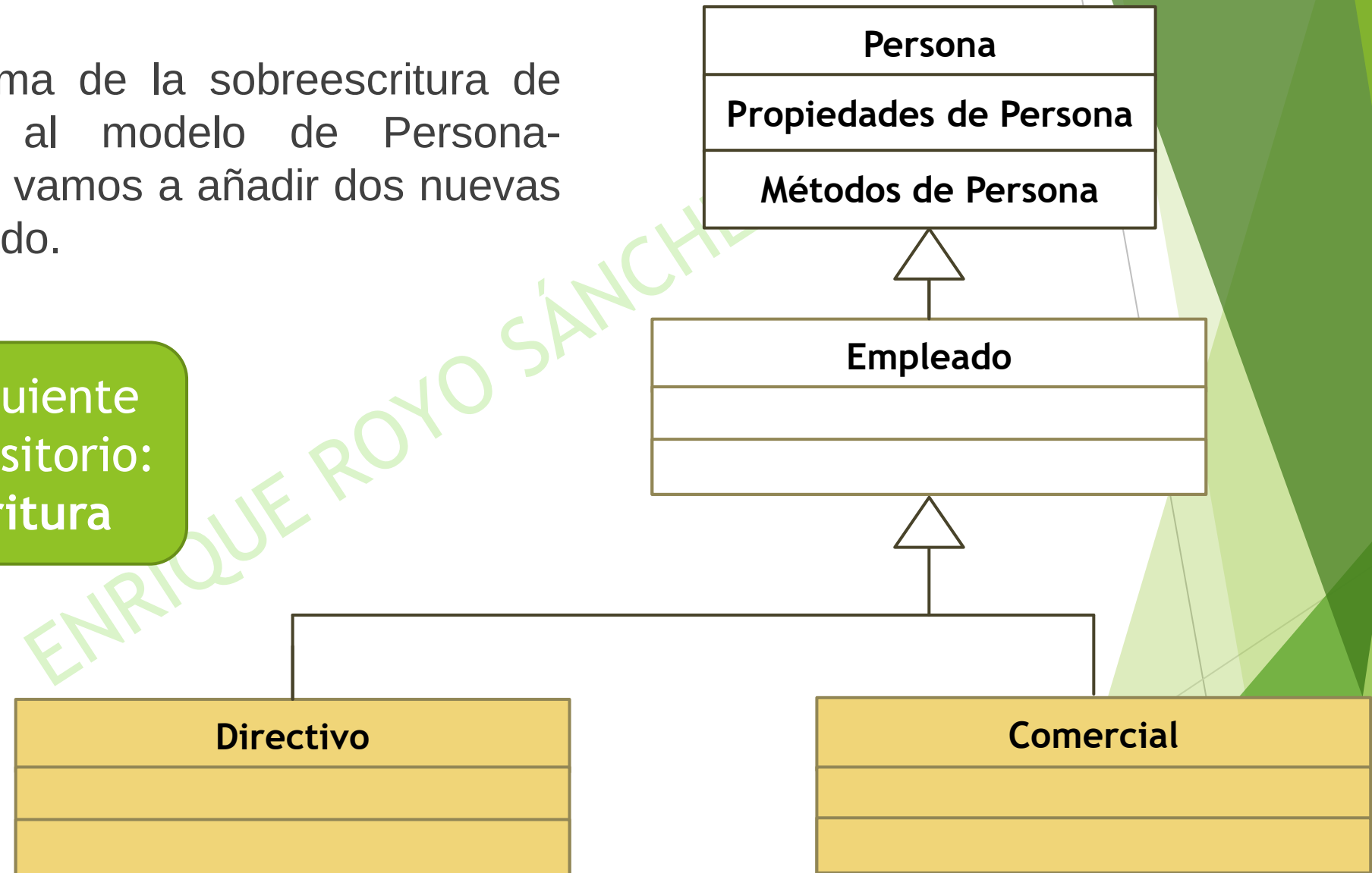
~~Código de la Superclase~~
Código de la Subclase

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- Para aprender el tema de la sobreescritura de métodos volvemos al modelo de Persona-Empleado y además, vamos a añadir dos nuevas subclases de Empleado.

Estudiamos el siguiente proyecto del repositorio:
U4.P2.Sobrescritura



UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- ▶ Como podéis observar en el código anterior se **mezclan los dos usos típicos de la herencia**:
 - ▶ **Ampliación de una clase, añadiendo nuevos métodos**: por ejemplo, la clase Director añade los métodos *setBonusAnual* y *getBonusAnual*, que no estaban en la clase Empleado.
 - ▶ **Modificación del comportamiento de los métodos existentes en una clase**: las clases Comercial y Director sobrescriben el comportamiento del método *calculaNomina*.



Realizamos los ejercicios 6 y 7 del boletín

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

► El operador instanceof

- Con tantas clases hijas, madres, abuelas, hermanas esto es un lío. Ahora nos gustaría poder preguntar a un objeto algo como “¿eres un ejemplo de la clase X?”

- El operador **instanceof** hace algo parecido. Veámoslo:

```
Persona p = new Persona();
```

```
String texto = “Mi texto”;
```

```
if (p instanceof Persona) ➡ devuelve true
```

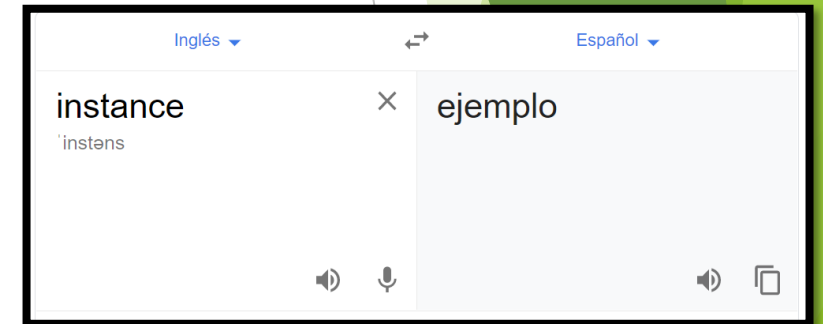
...

```
if (texto instanceof String) ➡ devuelve true
```

...

```
if (texto instanceof Persona) ➡ devuelve false
```

...



UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

► “Object está en todas partes”

- ¿Recuerdas cuando en la Unidad 1 decíamos que un principio de la programación orientada a objetos es que “**todo es un objeto**”?
- Pues los creadores de Java se lo tomaron al pie de la letra y crearon la clase **Object** que funciona como ancestro común de todas las clases. Todas las clases la tienen como superclase directa o indirectamente.
- Cuando creamos una clase que no hereda explícitamente de otra el **compilador le añade automáticamente** el texto en rojo:

```
public class Persona extends Object {...}
```

UD 1 - POO Y ELEMENTOS BÁSICOS DE JAVA 1 - Principios de la programación O.O.

► «Todo es un objeto»

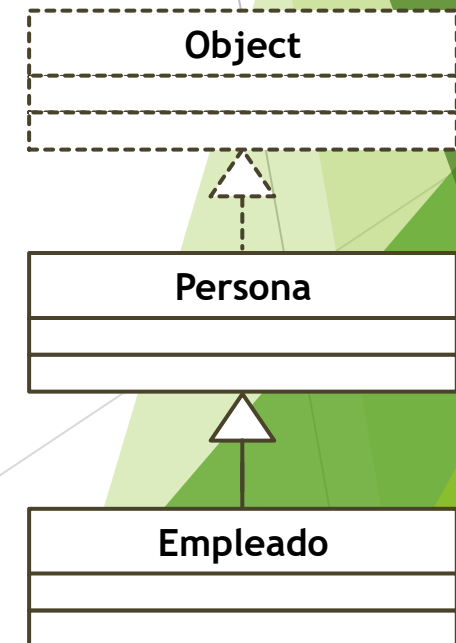
Los objetos se caracterizan por sus:

- Propiedades o atributos: *color, peso, nombre, apellidos, precio, altura...*
- Comportamientos o acciones que podemos realizar con ellos: *arrancar, parar, saltar, comer, abrir, cerrar, pagar, cobrar...*

Propiedades: color, textura, cantidad de aire...



Comportamientos: botar, rodar, girar...

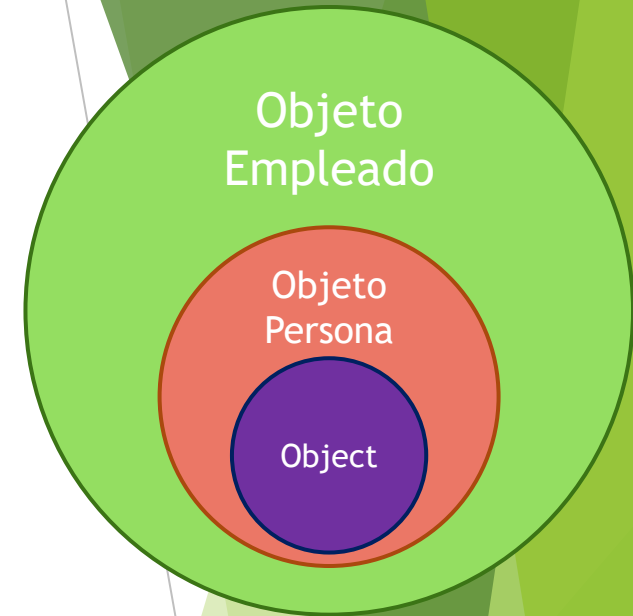


UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- ▶ Como, además decíamos que cuando un objeto hereda de otro es como si lo tuviera encerrado dentro, entonces todos los objetos contienen a Object en su interior... como decíamos: “**Object está en todas partes**”
- ▶ Fíjate entonces las respuestas que da el operador **instanceof** en el siguiente ejemplo:

```
Empleado emp = new Empleado();  
if (emp instanceof Empleado) // devuelve true  
    System.out.println("Soy Empleado");  
If (emp instanceof Persona) // devuelve true  
    System.out.println("También soy Persona");  
If (emp instanceof Object) // devuelve true  
    System.out.println("Y también soy un Object");
```



Los objetos se estructuran POR CAPAS como las CEBOLLAS

¿y tú de quién eres?



UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- ▶ Así que para Java solo existen dos tipos de datos:

Objetos (hijos legítimos del “dios Object”)	Tipos primitivos (pobres fuera del “paraíso de Object”)
String, Scanner, Persona, int[] ...	int, double, boolean...

- ▶ Vamos a echar un ojo a la clase **Object** para ver qué es lo que tiene dentro.

<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Object.html>

- ▶ Analicemos algunos de sus métodos:



*El “Dios Object”
está en todas
partes*

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- ▶ El método **clone** permite crear una copia exacta del objeto (no lo vamos a usar)
- ▶ Los métodos **wait**, **notify**, **notifyAll** se utilizan en programación multihilo para “dormir” y “despertar” a uno o varios hilos de ejecución (la programación multihilo es fascinante pero se sale del alcance de este curso).
- ▶ El método **getClass** devuelve un objeto de la clase Class que te devuelve información sobre la clase a la que pertenece un objeto (lo usaremos dentro de poco...)
- ▶ El método **hashCode** devuelve un entero que es como el “DNI” de un objeto en memoria. Esto sirve para almacenar objetos en tablas Hash (lo estudiaremos en la siguiente unidad didáctica)



UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- ▶ El método **equals (Object o)** recibe una referencia a un objeto y la compara con la referencia del objeto actual, devolviendo *true* si apuntan a la misma zona de memoria y *false* en caso contrario.
- ▶ El método **toString()** devuelve un texto que debe representar al objeto.
 - ▶ Este método suele ser **sobrescrito a menudo** incluyendo la información más relevante de las propiedades de un objeto o clase.
 - ▶ Además, este método se llama automáticamente cuando se quiere imprimir la representación textual de un objeto. Ejemplo:

```
Bombilla b = new Bombilla();
```

```
System.out.println(b); // Automáticamente se llama a b.toString()
```

¿a==b?

Estudiamos el código
U4.P3.DestripandoObject

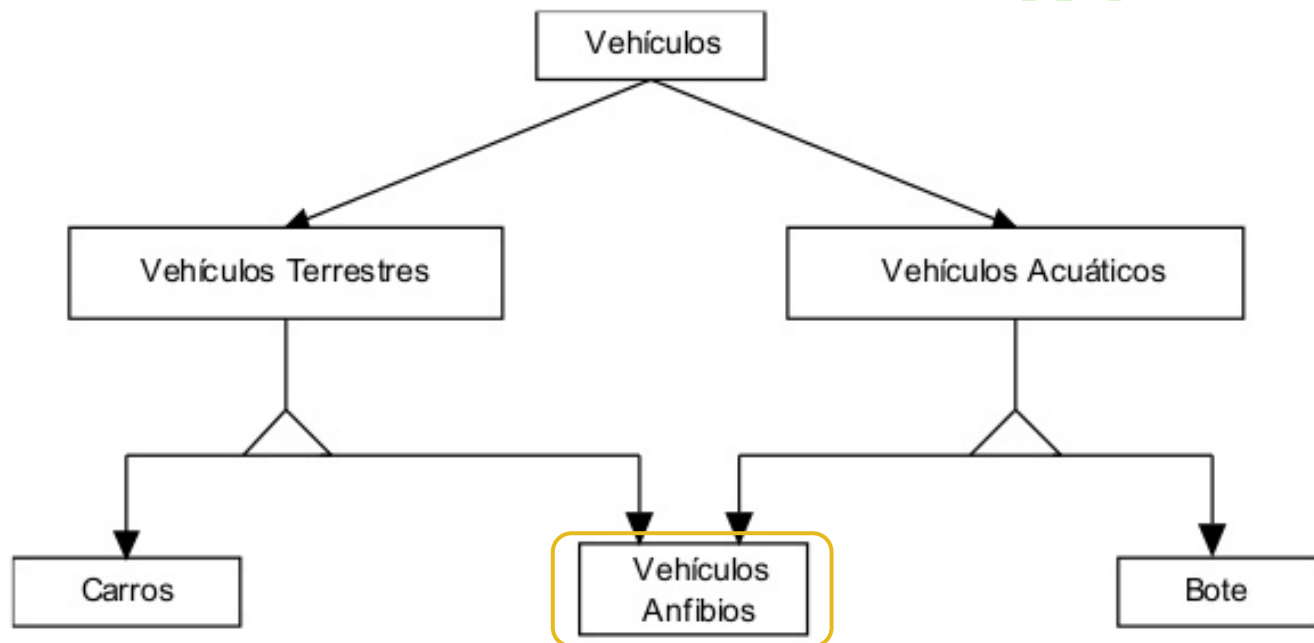
Realizamos el
ejercicio 8 del
boletín

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

► ¿Una clase puede tener dos padres/madres?

- Es decir, ¿una clase puede heredar de dos clases? ¿se permite la herencia múltiple?
- **Java no lo permite** pero hay otros lenguajes que sí. Esto permitiría resolver problemas como este:



Los objetos
tienen familias
monoparentales

Java no permite la herencia múltiple porque si hubiera un método común en ambas superclases (por ejemplo: `arrancar()`), el compilador no sabría qué código escoger...

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

► El modificador final en clases y métodos

- Si eres programador en una empresa y “un poquito hacker” puedes atacar el sistema “desde dentro” o extraer información de forma ilícita del siguiente modo:
 - Creas una **clase “impostora”** que **herede de otra clase “legítima”** propia del sistema y modificas el código para que se use la clase impostora en vez de la original.
 - La clase impostora tendría los mismos métodos que la original pero éstos podrían ser sobrescritos para que hagan “otras cosas adicionales”.
- Para solucionar esto, Java permite utilizar el **modificador final sobre clases y métodos** para aportar seguridad a nuestro código. Veamos cómo funciona:



FINAL

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- Si hacemos que una clase sea **final** entonces impedimos que se puedan sacar subclases de ella. La hacemos “estéril”. Sintaxis:

```
public final class ClaseEsteril {  
    ...  
}
```

- Si se intenta heredar de una clase declarada como **final**, el compilador lo marcará como un error y no se podrá generar el *bytecode* del programa.
- La API de Java está llena de clases declaradas como **final** para dotar de más seguridad al código generado.
 - Ejemplos: **String**, **Math**, **Scanner**...



UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- Otra opción menos radical de “securizar” nuestro código consiste en “esterilizar” algunos métodos pero permitiendo que la clase pueda tener hijos. Es decir, aplicamos **final** algunos métodos para que no puedan ser sobrescritos. Ejemplo:

```
public class ClaseFertil {  
    public final void esteMetodoNoSePuedeSobrescribir() {...}  
    ...  
}
```

```
public class ClaseHija extends ClaseFertil {  
    // Si lo intentamos sobrescribir el compilador nos da un error  
    public void esteMetodoNoSePuedeSobrescribir() {...}  
}
```



UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

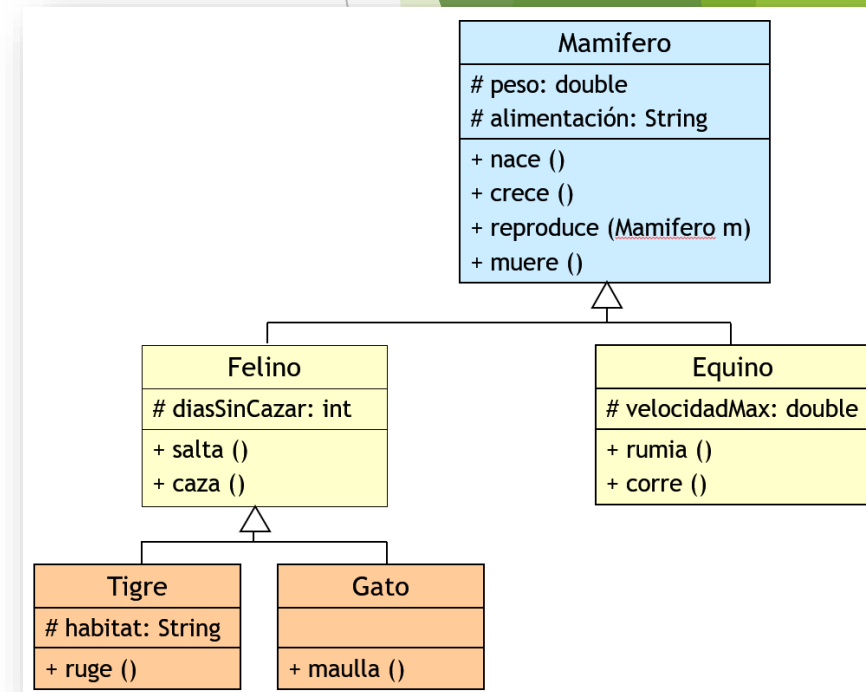
► El modificador *abstract* en clases y métodos

- Cuando usamos una jerarquía de clases, normalmente las clases superiores suelen ser conceptos abstractos que **no tienen una correspondencia con ningún objeto real**.

- **Ejemplo:** en el mundo real existen ejemplares de tigres o de gatos pero existe ningún ejemplar de “felino” o “mamífero”. Estos son conceptos que nos permiten identificar a un colectivo de seres vivos que tienen unas características comunes pero que no tienen “existencia física”.

- Entonces podría ser interesante “prohibir” la creación de objetos de ciertas clases porque son “demasiado abstractas”. Es decir, lo que queremos prohibir es:

Felino f = **new** Felino();
Mamifero m = **new** Mamifero;



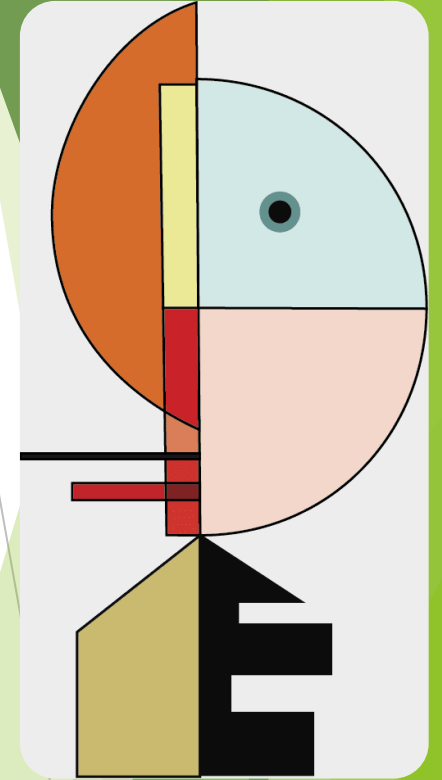
UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- El modificador **abstract** aplicado a clases impide que se puedan crear objetos a partir de ellas. La sintaxis es:

```
public abstract class Mamifero {  
    // Propiedades  
    // Métodos con su código  
}
```

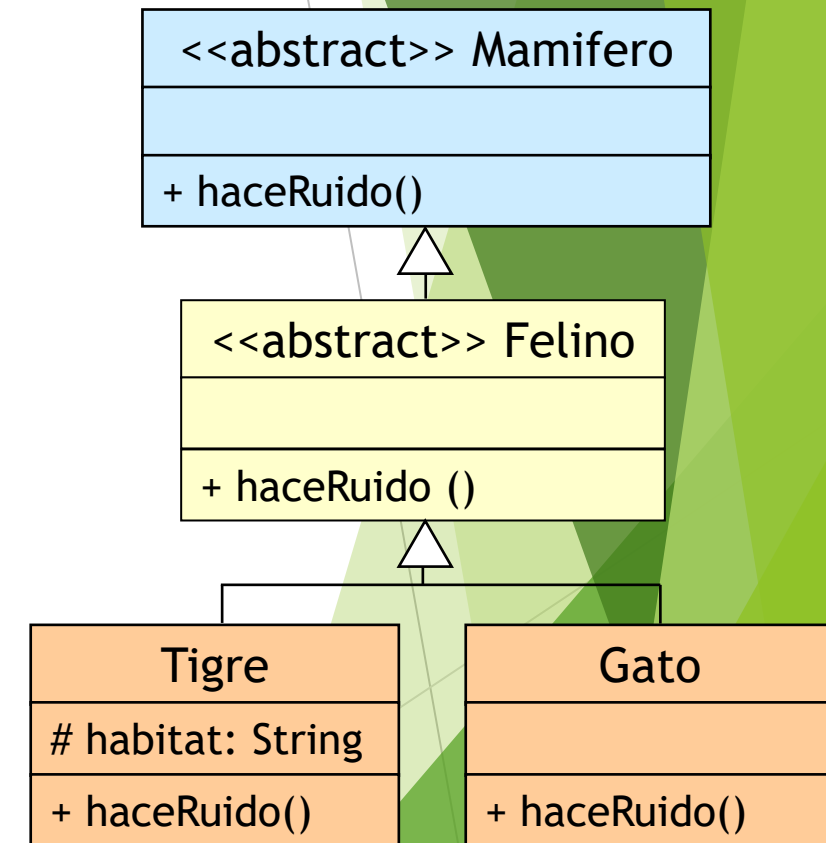
- Esta clase podría ser extendida por otras clases hijas pero **NO podríamos crear un objeto directamente** con un *new Mamifero()*.
- De este modo, si queremos usar el código escrito en la clase Mamifero **estamos obligados a sacar una subclase de ella**.
 - Esta nueva subclase también podrá ser abstracta (public abstract class Felino) o “concreta” (public class Tigre)



UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- ▶ Vamos a darle una última vuelta de tuerca a este concepto. ¿Cómo escribo el código de un método como *haceRuido()* cuando nunca crearemos un objeto de la clase Mamifero o de la clase Felino?
 - ▶ Sabemos que un gato hace “Miau” y un tigre hace “Roaaaar” pero ¿qué ruido hace un Mamifero?
 - ▶ Digamos que la clase Mamifero es tan abstracta que no tenemos información suficiente para escribir el código de *haceRuido()*
- ▶ Necesitamos un mecanismo de poder decirle al compilador que **NO vamos a escribir el código de un método porque queremos que lo hagan las subclases.**



UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- Esto se hace mediante el modificador **abstract** aplicado a **métodos**. Veamos la sintaxis:

```
public abstract class Mamifero {
```

```
// ...
```

```
public abstract void haceRuido();
```

```
}
```

- Los métodos abstractos terminan en ; y no llevan { }
- Si una clase **tiene 1 o más métodos abstractos entonces debe ser declarada como abstracta**.
- Sin embargo, pueden existir clases declaradas como abstractas pero que no contengan ningún método abstracto.

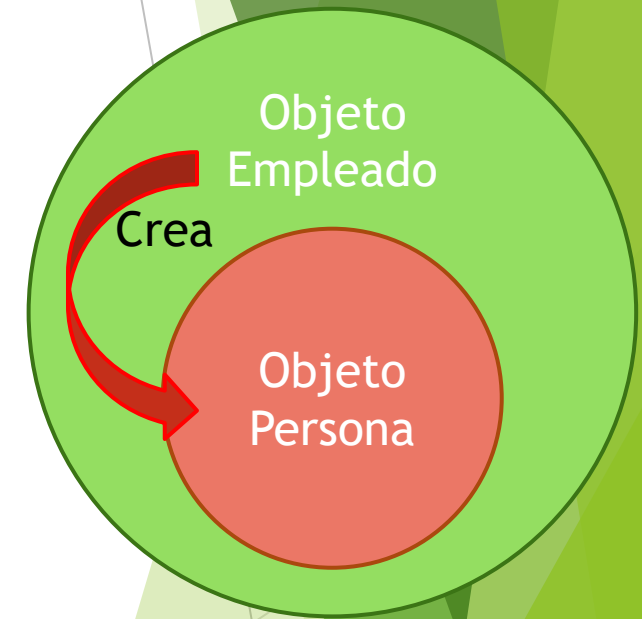
Realizamos los ejercicios 9-10 del boletín

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

► La herencia y los constructores

- Sabemos que en la herencia tenemos un objeto de la superclase “encerrado” en otro objeto de la subclase, entonces **¿cómo se crean ambos objetos? ¿creamos primero a la Persona y después al Empleado? ¿O al revés?**
- Siempre crearemos los objetos usando el constructor de la subclase (Empleado) y este constructor será **el responsable** de crear el objeto de la superclase (Persona).
- El tema de los constructores y la herencia es “delicado” porque el lenguaje Java **“nos intenta ayudar a su manera”** insertando cierto código automáticamente, que más de una vez nos lía más que otra cosa. Veamos cómo se hace todo el proceso.



UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- Tomemos un ejemplo sencillo:

```
public class Persona {  
    private String nombre, apellidos;  
  
    public Persona() {  
        nombre = "";  
        apellidos = "";  
    }  
    // getters y setters  
}
```

El objeto Empleado llama al constructor de la superclase como primera línea.
Si el programador no la escribe, Java la pone por nosotros.

```
public class Empleado extends Persona {  
    private final double SALARIO_MINIMO = 1050;  
    private double salario;  
  
    public Empleado() {  
        // La primera línea del constructor de la subclase debe  
        // ser una llamada al constructor de la superclase.  
        // Si el desarrollador no la pone explícitamente, Java  
        // añadiría esta línea:  
        super();  
        salario = SALARIO_MINIMO;  
    }  
}
```

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- Los problemas se producen cuando la versión del constructor que se llama desde la subclase **NO EXISTE** en la superclase. Ejemplo:

```
public class Persona {  
    private String nombre, apellidos;  
  
    public Persona(String nombre, String apellidos) {  
        this.nombre = nombre;  
        this.apellidos = apellidos;  
    }  
    // getters y setters  
}
```

**No existe el constructor
public Persona()**

Recordamos que Java sólo lo crea automáticamente si el desarrollador no crea ningún constructor.

```
public class Empleado extends Persona {  
    private final double SALARIO_MINIMO = 1050;  
    private double salario;  
  
    public Empleado() {  
        // La primera línea del constructor de la subclase debe  
        // ser una llamada al constructor de la superclase.  
        // Si el desarrollador no la pone explícitamente, Java  
        // añadiría esta línea:  
        super();  
        salario = SALARIO_MINIMO;  
    }  
}
```

UD 4 - RELACIONES ENTRE CLASES

3 - Herencia

- La solución en estos casos es que el desarrollador escriba explícitamente la versión correcta del constructor de la superclase:

```
public class Persona {  
    private String nombre, apellidos;  
  
    public Persona(String nombre, String apellidos) {  
        this.nombre = nombre;  
        this.apellidos = apellidos;  
    }  
    // getters y setters  
}
```

Tenemos que escribir explícitamente la versión a usar.
Si no lo hacemos, Java insertaría automáticamente la línea `super()` y fallaría.

Descargamos y probamos
U4.P4.HerenciaConstructores

```
public class Empleado extends Persona{  
    private final double SALARIO_MINIMO = 1050;  
    private double salario;  
  
    public Empleado(String nombre, String apellidos) {  
        // Ahora debemos escribir explícitamente la llamada  
        // a la versión correcta del constructo de la  
        // superclase:  
        super(nombre, apellidos);  
        salario = SALARIO_MINIMO;  
    }  
}
```

Realizamos el
ejercicio 11 del
boletín

UD 4 - RELACIONES ENTRE CLASES

4 - Polimorfismo

► 4 – Polimorfismo

- Sabemos que la herencia de clases nos permite reutilizar nuestro código ampliándolo y modificándolo. Pero además la herencia nos aporta otra ventaja más: **el polimorfismo**.
- Algo **polimorfo** es algo que se puede presentar de varias formas. De ahí las imágenes con la plastilina...
- También podríamos hablar de que cada uno de nosotros “somos polimorfos” en el siguiente sentido: yo soy profesor/a para mis alumnos/as. Contribuyente para Hacienda. Arrendador/a para mi inquilino. Conductor/a para la DGT. Padre/Madre para mis hijas. Hijo/a para mis padres...
- Este es el sentido de polimorfismo que vamos a aplicar sobre una jerarquía de clases.



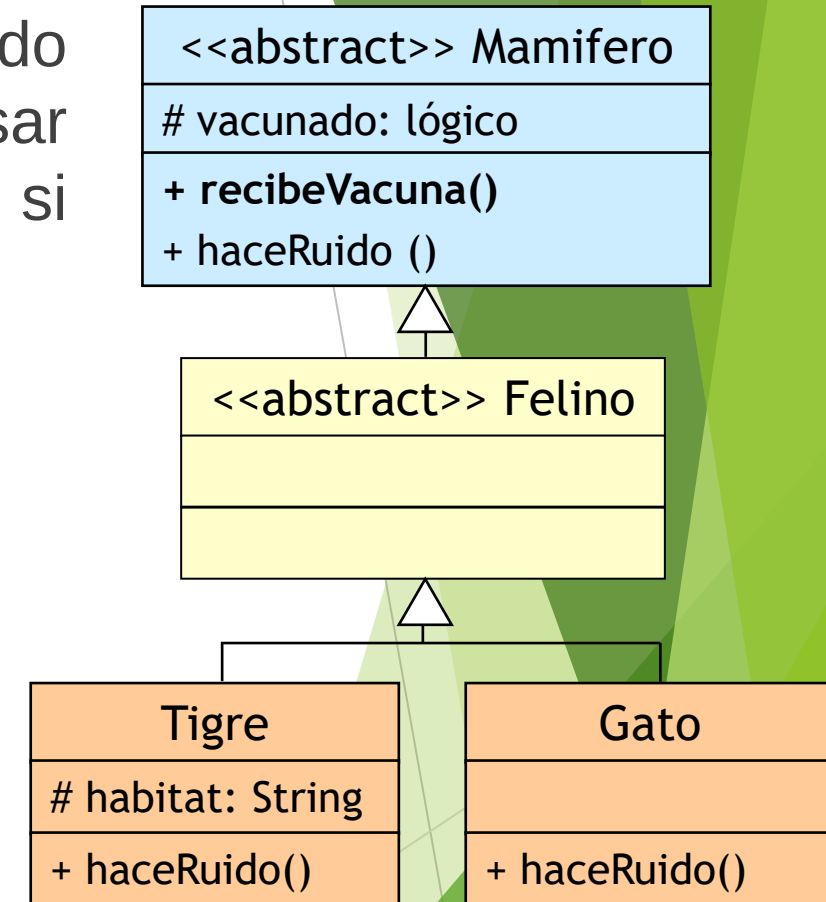
UD 4 - RELACIONES ENTRE CLASES

4 - Polimorfismo

- En una jerarquía de clases, las clases se relacionan formando una especie de “familia”. Y, en ocasiones, nos puede interesar tratar a todos los miembros de la familia por igual, sin importar si es “el abuelo” o “un nieto”... Observa el siguiente código:

Un veterinario tiene que ponerle la vacuna a todos los Mamíferos de un zoo, sin importar de qué tipo sean.

```
public class Veterinario {  
    public static void main(String[] args) {  
        Tigre t1 = new Tigre();  
        Tigre t2 = new Tigre();  
        Gato g1 = new Gato();  
        Gato g2 = new Gato();  
  
        Mamifero[] animales = new Mamifero[4];  
        animales[0] = t1;  
        animales[1] = t2;  
        animales[2] = g1;  
        animales[3] = g2;  
  
        for(Mamifero a : animales)  
            a.recibeVacuna();  
    }  
}
```



Código disponible en
U4.P5.PolimorfismoAnimal

UD 4 - RELACIONES ENTRE CLASES

4 - Polimorfismo

- ▶ En este código hay varias cosas interesantes:
 - ▶ Se crea un array de referencias a Mamifero, aunque sabemos que esta clase es abstracta y que no podremos crear objetos de ella.
 - ▶ Llenamos el array con objetos Tigre y Gato porque heredan de Mamifero, de modo que “saben comportarse como Mamifero”, porque estos objetos son Tigre/Gato pero también son Felino y Mamifero.
 - ▶ **Esto es el polimorfismo**, un objeto puede tener el comportamiento propio de la clase a la que pertenece pero también puede comportarse como un objeto de cualquiera de sus superclases.
 - ▶ Tener un array de Mamifero en vez de dos arrays, uno de Tigres y otro de Gatos, es **una gran ventaja** porque nos permite aplicar un tratamiento común (vacuna) a todos los objetos de forma cómoda. Imagina el problema que generaría no poder usar el polimorfismo y que hubiera 20 subclases.

```
public class Veterinario {  
    public static void main(String[] args) {  
        Tigre t1 = new Tigre();  
        Tigre t2 = new Tigre();  
        Gato g1 = new Gato();  
        Gato g2 = new Gato();  
  
        Mamifero[] animales = new Mamifero[4];  
        animales[0] = t1;  
        animales[1] = t2;  
        animales[2] = g1;  
        animales[3] = g2;  
  
        for(Mamifero a : animales)  
            a.recibeVacuna();  
    }  
}
```

*A los ojos del
veterinario todos los
objetos tigres/gatos se
comportan como
Mamiferos*

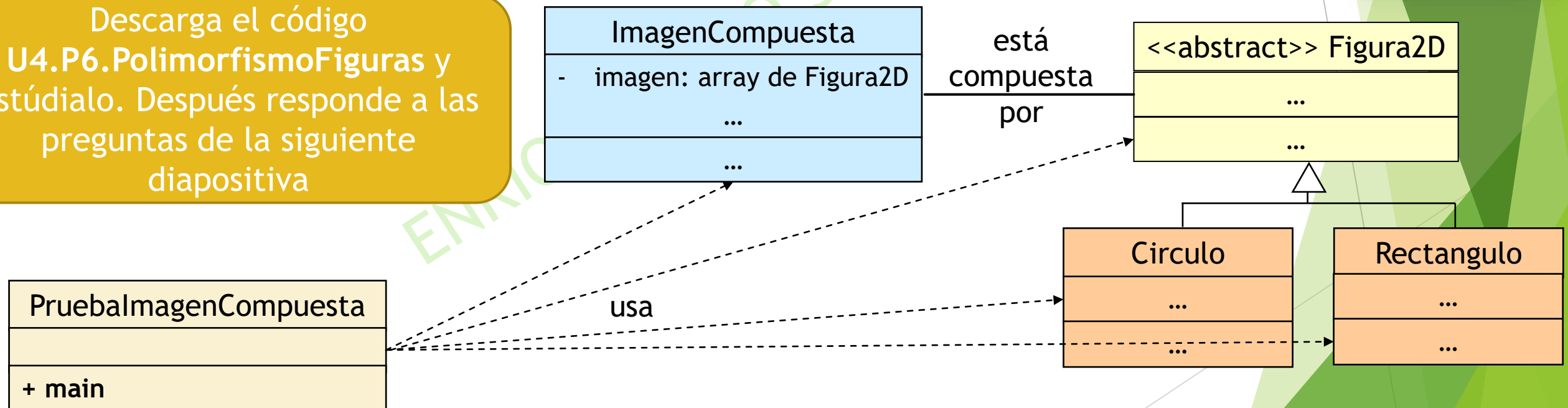
UD 4 - RELACIONES ENTRE CLASES

4 - Polimorfismo

- Es como si el veterinario se pusiera unas **gafas que solo permiten ver los objetos de tipo Mamifero**. Con esas gafas da igual si estamos mirando un gato o un tigre, ambos los vemos como mamíferos.
- Vamos a poner otro ejemplo con el siguiente diagrama UML:



Descarga el código
U4.P6.PolimorfismoFiguras y
estúdialo. Después responde a las
preguntas de la siguiente
diapositiva



UD 4 - RELACIONES ENTRE CLASES

4 - Polimorfismo

- ▶ ¿Qué significa que la clase Figura2D sea *abstract*?
- ▶ ¿Qué métodos con código heredan las subclases?
- ▶ ¿Qué métodos sin código heredan las subclases? ¿Qué tienen que hacer con estos métodos?
- ▶ ¿Qué significa que las clases Circulo y Rectángulo sean *final*?
- ▶ ¿Existe en una versión del constructor sin parámetros en la clase Circulo? ¿Y en la clase Figura2D?
- ▶ Explica la primera línea de código del constructor de la clase Circulo?
- ▶ Explica cómo la clase ImagenCompuesta trabaja con un array de Figura2D si esta clase es abstracta y no podemos crear objetos de ella.
- ▶ Si no existiera el polimorfismo qué propiedades tendría que tener la clase ImagenCompuesta para poder trabajar con las distintas figuras.

Realizamos el
ejercicio 12 del
boletín

UD 4 - RELACIONES ENTRE CLASES

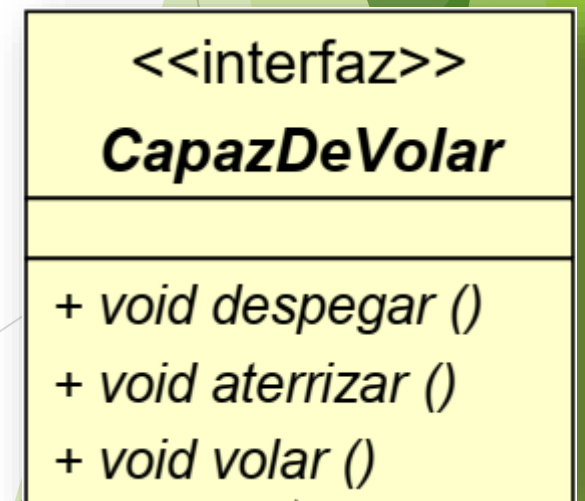
5 - Interfaces

► 5 – Interfaces

- Recordemos que las clases son las plantillas o moldes con las que podemos crear los objetos.
- Pues bien, **las interfaces son las plantillas o moldes con las que podemos crear clases...** Es el molde del molde.
- Cuando queremos crear una clase utilizando una interfaz como plantilla, entonces decimos que la “**clase implementa la interfaz**”.
- Veamos un ejemplo de interfaz:

```
public interface CapazDeVolar {  
    void despegar();  
    void aterrizar();  
    void volar();  
}
```

Todos los métodos son considerados como **public abstract**. Estos modificadores pueden omitirse, o escribirse también, si se desea.



UD 4 - RELACIONES ENTRE CLASES

5 - Interfaces

- ▶ Normalmente, las interfaces expresan “capacidades”. Ejemplos: Transportable, Almacenable, PuedeSaltar, CapazDeVolar, Imprimible, Comestible, Vendible...
- ▶ Ahora veamos cómo creamos una clase a partir de la interfaz anterior:

```
public class Helicóptero implements CapazDeVolar {
```

```
    // ... Propiedades del helicóptero
```

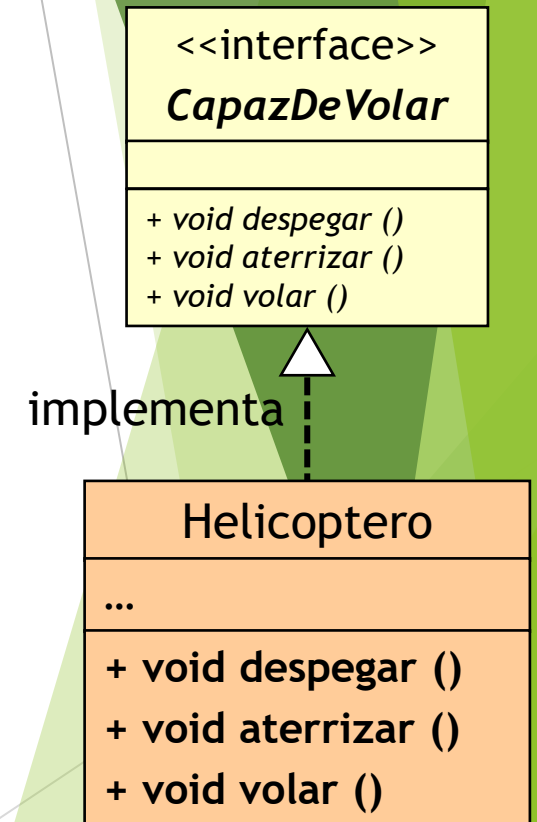
```
    public void despegar() { /* Código aquí */ }
```

```
    public void aterrizar() { /* Código aquí */ }
```

```
    public void volar() { /* Código aquí */ }
```

```
    // ... Otros métodos
```

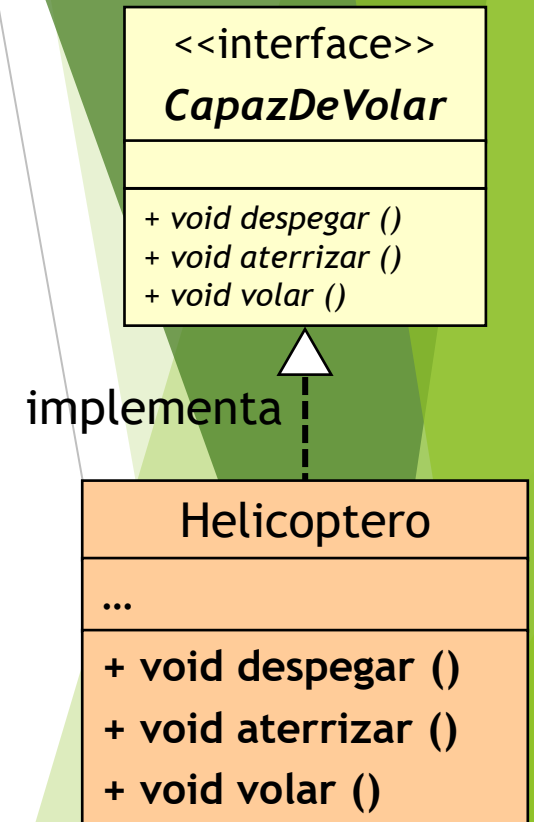
```
}
```



UD 4 - RELACIONES ENTRE CLASES

5 - Interfaces

- ▶ Dado que la clase Helicóptero la hemos creado tomando la interfaz como plantilla, esto **nos obliga a escribir código para cada uno de los métodos declarados en la interfaz**.
- ▶ La interfaz **se comporta como un “contrato”** que deben cumplir las clases que implementen dicha interfaz. El contrato de la interfaz podría decir algo como:
 - ▶ “La clase que implemente la interfaz tiene la OBLIGACIÓN de escribir código para todos los métodos de la interfaz”
 - ▶ “La clase que implemente la interfaz tiene DERECHO a escribir el código como le apetezca y crear tantos métodos adicionales como desee”
- ▶ Las interfaces son una herramienta del lenguaje con mucha capacidad expresiva. Permiten que un arquitecto/diseñador comunique a su equipo de desarrollo las capacidades que deben tener las clases del sistema. Existe una técnica de diseño de software “orientada a interfaces”, que las utiliza como “ladrillo principal”.

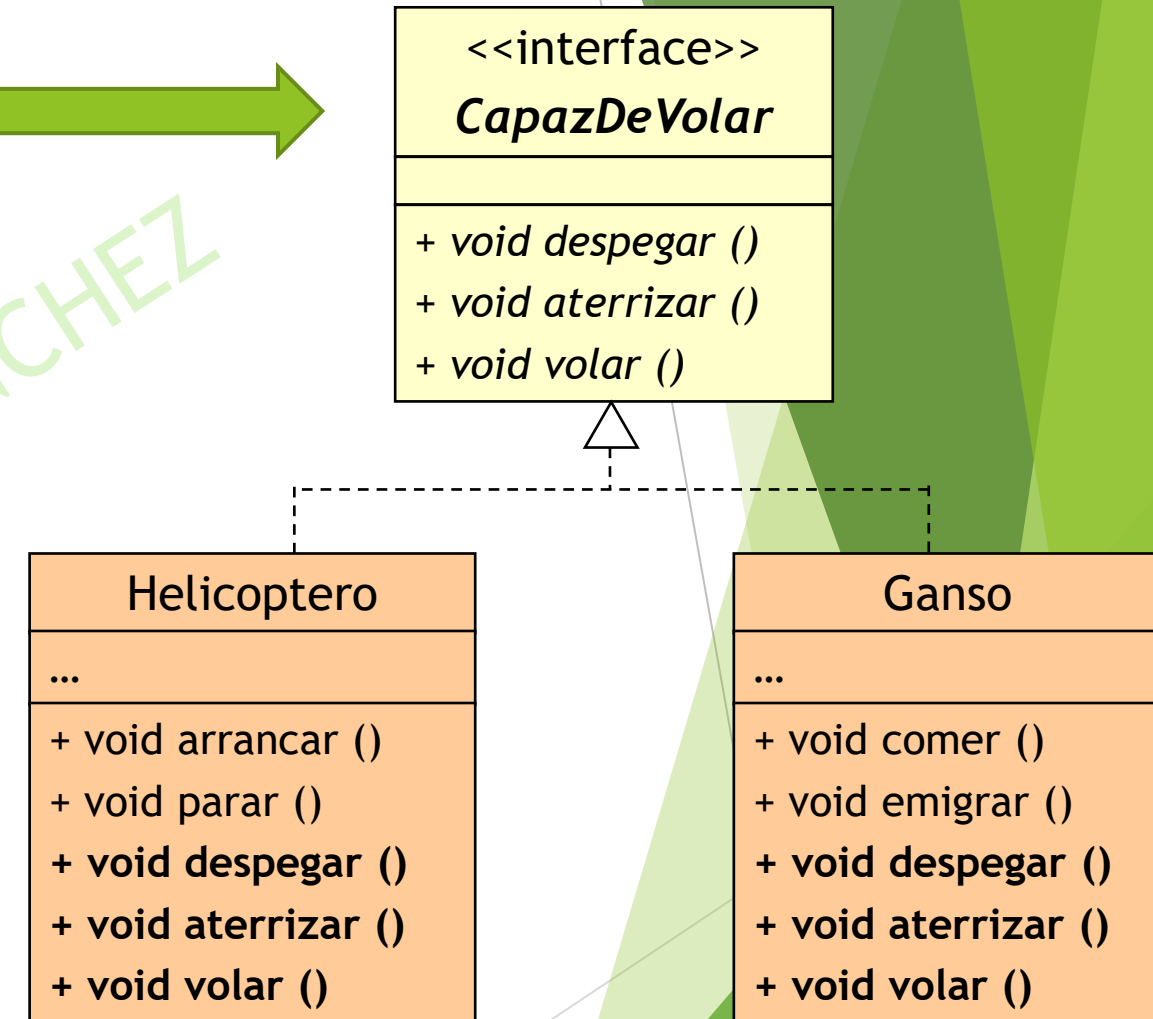
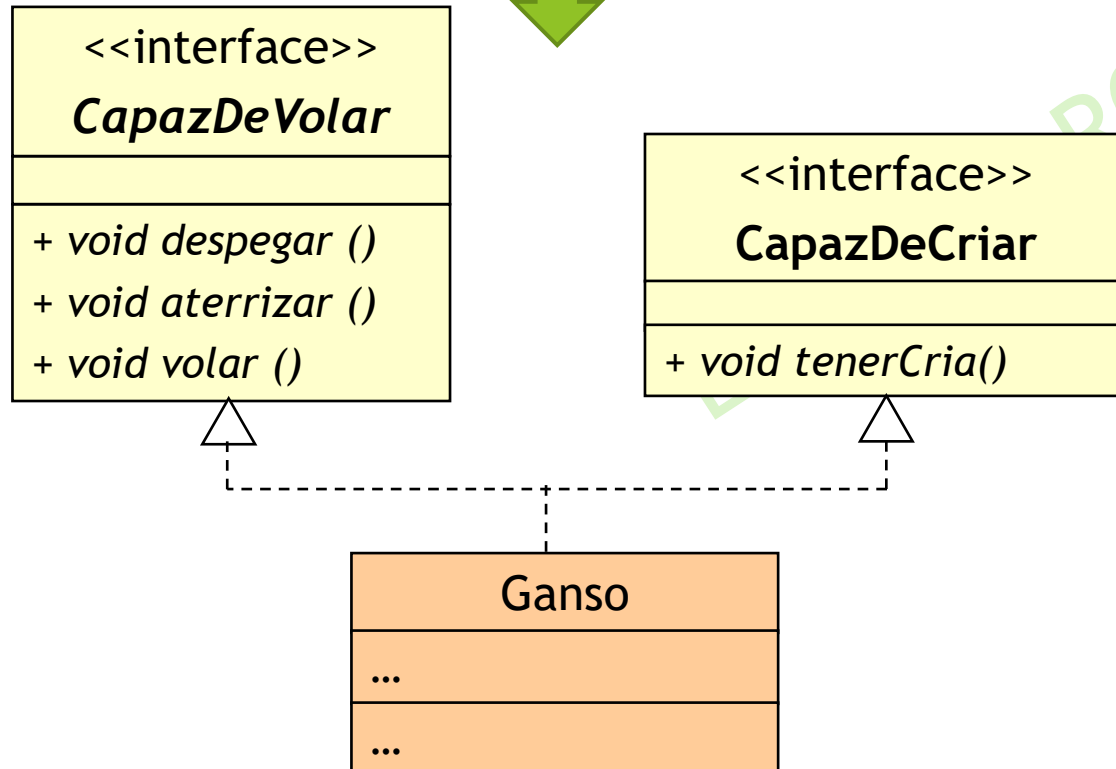


Lee el ejercicio
13 del boletín

UD 4 - RELACIONES ENTRE CLASES

5 - Interfaces

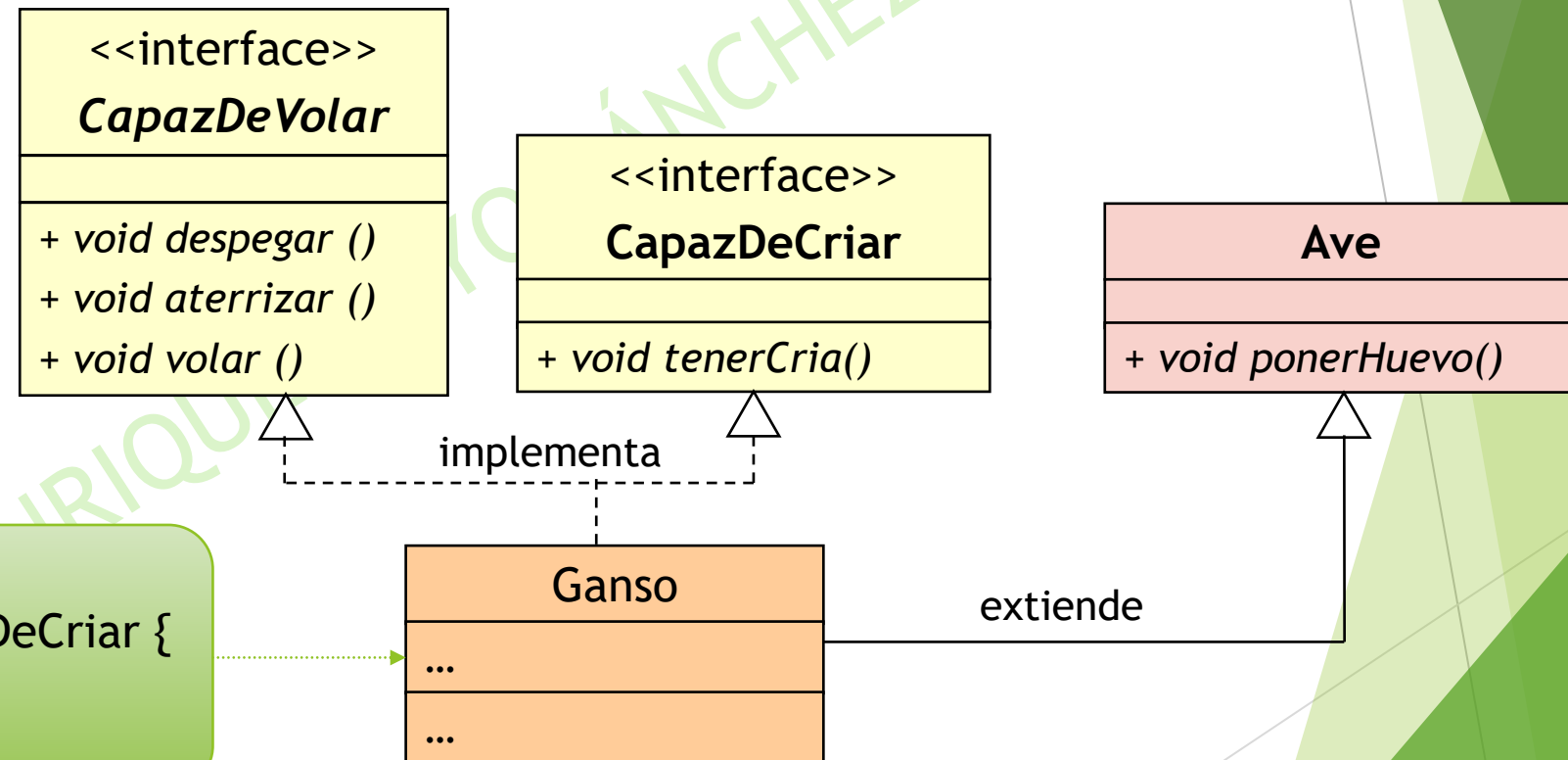
- ▶ Varias clases pueden implementar la misma interfaz.
- ▶ Y también una clase puede implementar más de una interfaz



UD 4 - RELACIONES ENTRE CLASES

5 - Interfaces

- **OJO:** Una clase puede implementar más de una interfaz **pero sólo puede heredar de una superclase.**



UD 4 - RELACIONES ENTRE CLASES

5 - Interfaces

- ▶ Las interfaces normalmente solo llevan métodos (públicos y abstractos) pero también pueden llevar propiedades que serán tratadas como constantes estáticas y públicas. Es decir, todas serán declaradas automáticamente como **public static final**.

- ▶ Ejemplo:

```
public interface CapazDeVolar {  
    double GRAVEDAD = 9.8;  
    void despegar();  
    void aterrizar();  
    void volar();  
}
```



UD 4 - RELACIONES ENTRE CLASES

5 - Interfaces

- Además, el **polimorfismo también funciona con interfaces** permitiendo que objetos de muy distinta índole puedan ser tratados como un grupo. Ejemplo:

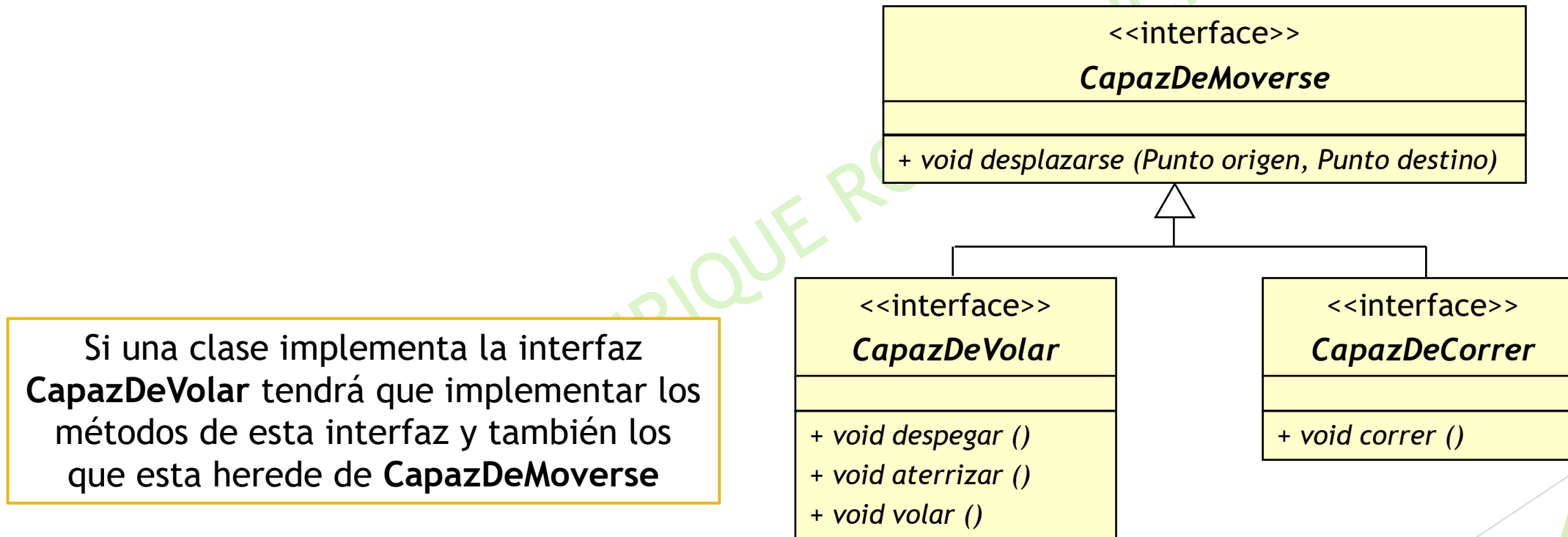
...

```
CapazDeVolar[ ] voladores = new CapazDeVolar [3];  
voladores[0] = new Helicoptero();  
voladores[1] = new Ganso();  
voladores[2] = new Helicoptero();  
for (CapazDeVolar v: voladores) {  
    v.despegar();  
}
```

UD 4 - RELACIONES ENTRE CLASES

5 - Interfaces

- Y para rizar el rizo: **también existe la herencia entre interfaces.** Eso sí, sigue las mismas reglas que la herencia entre clases, es decir, una interfaz solo puede heredar de una y solo una “superinterfaz”.



En las siguientes unidades manejaremos algunas interfaces de las librerías del lenguaje y le veremos más sentido a este elemento

¿Te atreves con el RETO del boletín?

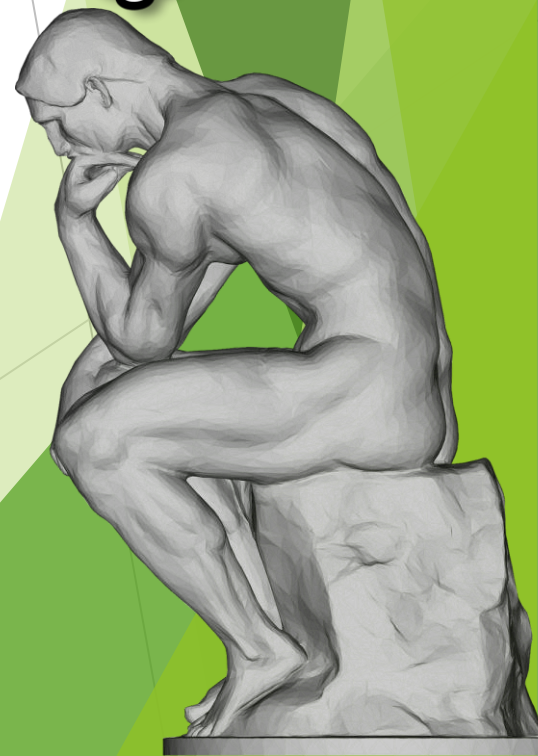
Realizamos los ejercicios del 14 al 18 del boletín

UD 4 - RELACIONES ENTRE CLASES

Cierre de la unidad

- ▶ En esta unidad hemos recordado que en el mundo del software **TODO CAMBIA** y que necesitamos preparar nuestro código para que **“abrace el cambio”** y no luche contra él.
- ▶ Mediante la composición podemos hacer que dos o más objetos **colaboren entre sí**.
- ▶ Con la herencia podemos **reutilizar funcionalidad** ya escrita previamente.
- ▶ El polimorfismo permite dar un **tratamiento uniforme a un conjunto de objetos de tipos distintos pero que tienen algo en común**, esto permite que nuestro código se adapte al cambio con mayor facilidad.
- ▶ Las interfaces nos permiten especificar un **"contrato de capacidades"** que podrá ser satisfecho por una o más clases.
- ▶ Seguiremos trabajando todas estas técnicas durante el resto del curso, poniéndolas en distintos contextos y profundizando en ellas.

Abrazar el cambio... eso da que pensar



UD 4 - RELACIONES ENTRE CLASES



CREATED BY KRASTEVOLICRADIOGRAPH